NIALL CASSIDY

SOFTWARE DEVELOPMENT

A2 WJEC COMPUTER SCIENCE

NIALL CASSIDY CANNDIDATE NO: 0109
CENTER NO: 71741

Section 6: Software Development (25/100) marks

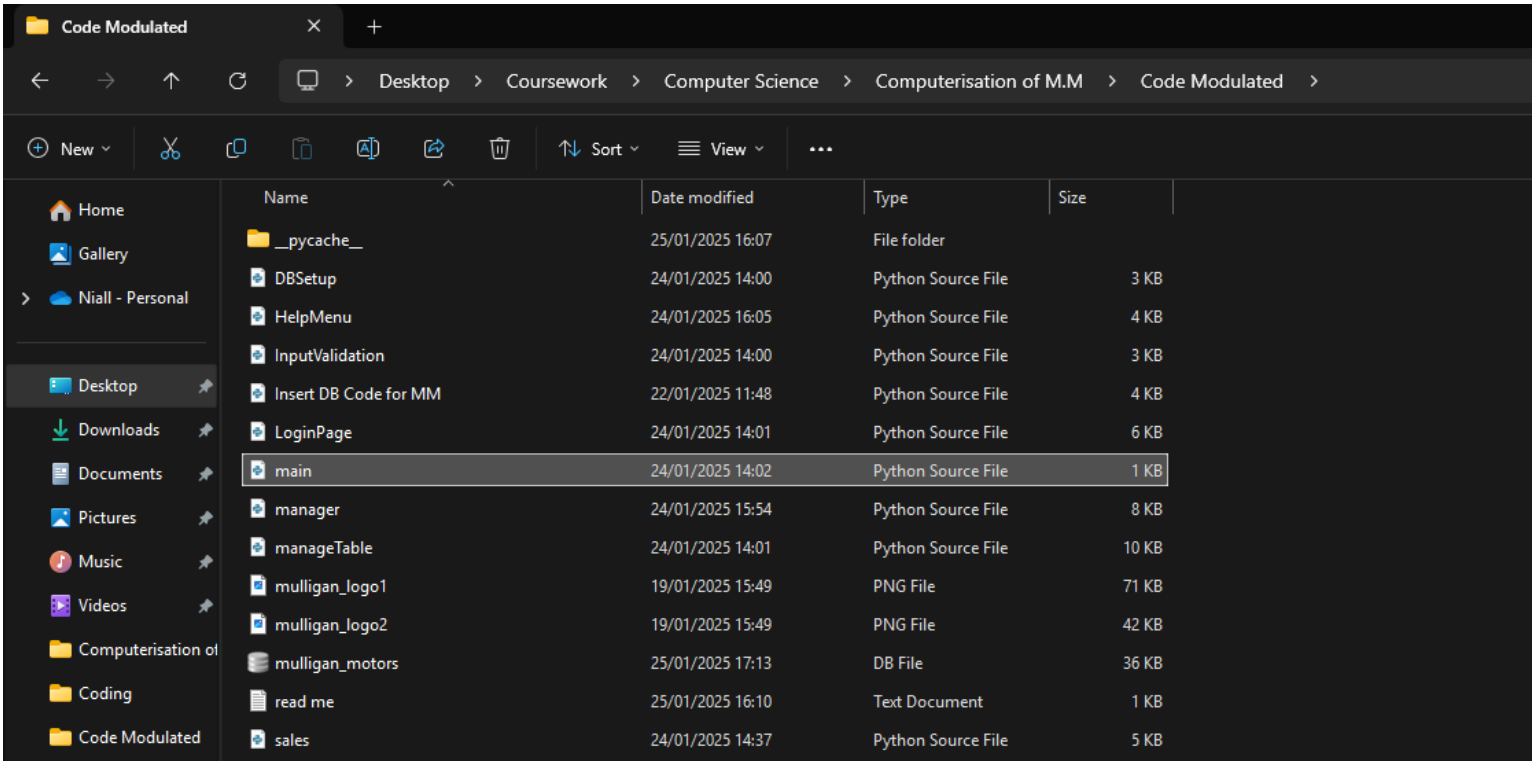
- **taken full account of the revised detailed design and refined the prototype in light of feedback and changed designs**
- **produced a functioning solution to a highly demanding problem that meets most of the objectives for the solution to the problem. Better solutions will meet almost all of the objectives for the proposed solution.**
- **used and fully exploited the programming facilities of the language**
- **demonstrated a sound understanding of the appropriate tools and techniques available to them**
- **created a well-structured data model that aids efficient data handling. Better candidates will present a data model normalised to third normal form.**
- **produced a solution that is well-structured and modular in nature. The solution makes good use of local variables and minimises the use of global variables.**
- **written code that is fully self-documenting and well-structured with annotation that will allow a competent third party to maintain the solution in future**
- **made use of complex user-defined routines. Better candidates will have made effective use of recursive algorithms.**
- **produced evidence of the effective use of validation for all key components. Better candidates will have created efficient routines for exception handling.**
- **fully documented the variables and actual data structures used to create the solution to the chosen problem**
- **included evidence of the completed user interface including a full description of the features that make it fit for audience and purpose**

6.0 File structure/layout

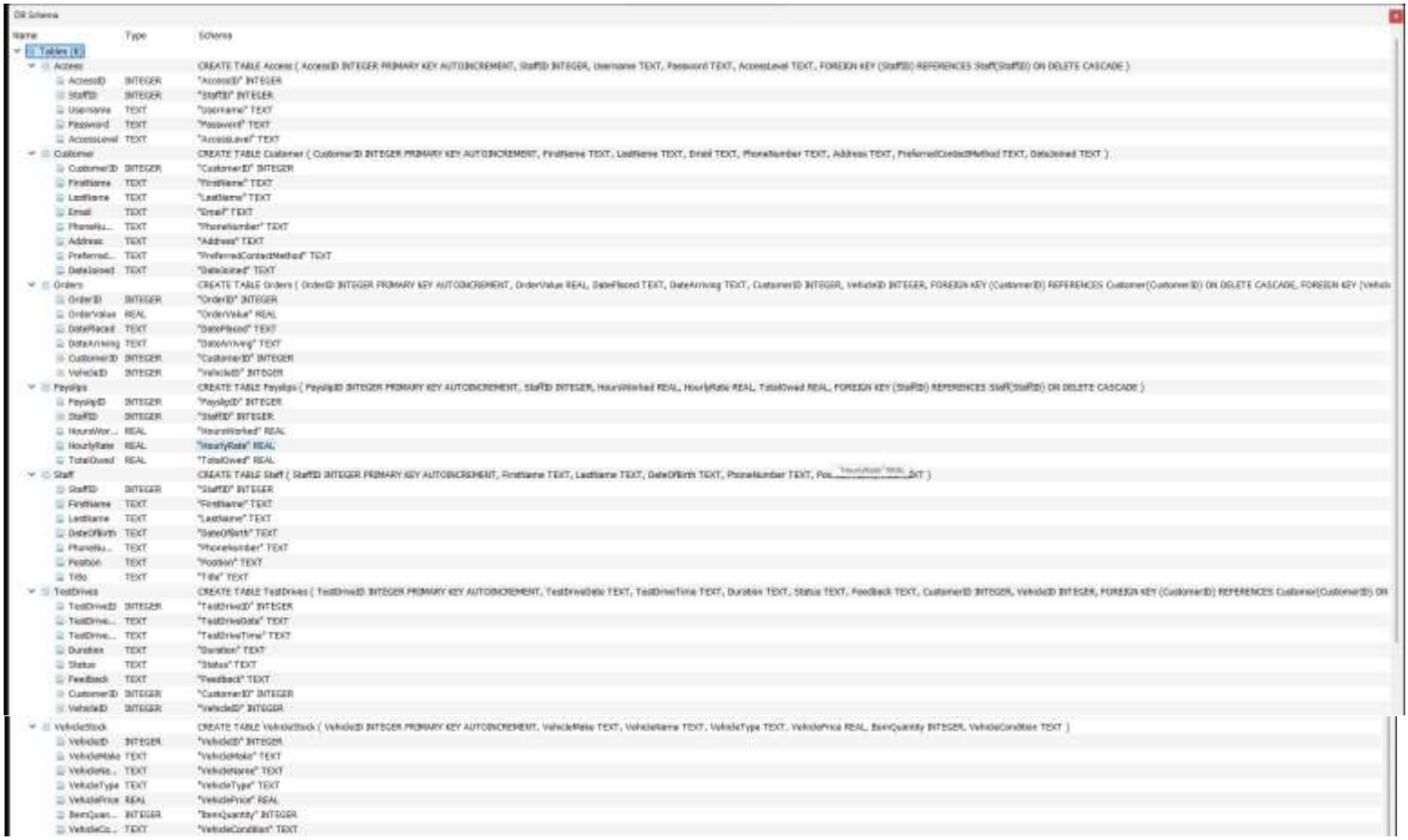
My final solution consists of 13 files (excluding __pycache__) :

- 9 Python files
- 2 .png image files
- 1 database file
- 1 .txt file

Whilst all code in the prototype was in one single python file, I decided to make the final version of Mulligan's Motors program modular in nature. This allows for easier and convenient maintenance for both me and a third party in the future especially as the program nears a thousand lines of code. All files are appropriately named and are self-identifying. E.g 'Validation.py' defines all the validation methods used and 'manageTable.py' defines the processes used to manipulate and manage the database information.



6.1 Database Structure



Using the ‘DB Browser for SQLite3’ I can view the schema for the program which I have screenshotted and showcased here.

I can also show the example data stored within the database below.

Access Table

	AccessID	StaffID	Username	Password	AccessLevel
	Filter	Filter	Filter	Filter	Filter
1	1	25	admin	root	admin
2	2	26	Salesboy	sales	sales
3	3	1	123	123	sales
4	4	1	alice_admin	adminPass	admin
5	5	2	bob_sales	salesPass	sales

Customer Table

	CustomerID	FirstName	LastName	Email	PhoneNumber	Address	PreferredContactMethod	DateJoined
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	John	Doe		1234567890	123 Elm Street	Phone	2025-01-10
2	2	Jane	Smith		9876543210	456 Oak Avenue	Email	2025-01-15

Order Table

	OrderID	OrderValue	DatePlaced	DateArriving	CustomerID	VehicleID
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	20000.0	2025-01-12	2025-01-20	1	1
2	2	18000.0	2025-01-14	2025-01-25	2	2

Payslip Table

	PayslipID	StaffID	HoursWorked	HourlyRate	TotalOwed
	Filter	Filter	Filter	Filter	Filter
1	1	1	40.0	20.0	800.0
2	2	2	35.0	15.0	525.0

Staff Table

	StaffID	FirstName	LastName	DateOfBirth	PhoneNumber	Position	Title
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Alice	Johnson	1985-05-05	5551112222	Manager	Ms.
2	2	Bob	Williams	1990-03-02	5553334444	Sales Rep	Mr.

Test-Drive Table

	TestDriveID	TestDriveDate	TestDriveTime	Duration	Status	Feedback	*CustomerID*	*VehicleID*
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	2025-02-01	10:00	30 minutes	Completed	Loved it!	1	2
2	2	2025-02-05	14:00	45 minutes	Pending		2	3

Vehicle Stock

	VehicleID	VehicleMake	VehicleName	VehicleType	VehiclePrice	ItemQuantity	VehicleCondition
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Toyota	Corolla	Sedan	20000.0	5	Slightly Used
2	2	Honda	Civic	Sedan	18000.0	3	Pristine
3	3	Ford	F-150	Truck	35000.0	2	Heavily Used

6.2 Code

Beneath is the code used in each of the 9 python files and their output in the program.

DB Setup

This module initialises the database and ensures that all entities and attributes that Mulligan Motors expects is present in the system.

```
5   def initialize_database():
6       conn = sqlite3.connect('mulligan_motors.db')
7       cursor = conn.cursor()
8       cursor.execute("PRAGMA foreign_keys = ON")
9
10      # VehicleStock
11      cursor.execute('''
12      CREATE TABLE IF NOT EXISTS VehicleStock (
13          VehicleID INTEGER PRIMARY KEY AUTOINCREMENT,
14          VehicleMake TEXT,
15          VehicleName TEXT,
16          VehicleType TEXT,
17          VehiclePrice REAL,
18          ItemQuantity INTEGER,
19          VehicleCondition TEXT
20      )''')
21
22      # Customer
23      cursor.execute('''
24      CREATE TABLE IF NOT EXISTS Customer (
25          CustomerID INTEGER PRIMARY KEY AUTOINCREMENT,
26          FirstName TEXT,
27          LastName TEXT,
28          Email TEXT,
29          PhoneNumber TEXT,
30          Address TEXT,
31          PreferredContactMethod TEXT,
32          DateJoined TEXT
33      )''')
34
35      # Orders
36      cursor.execute('''
37      CREATE TABLE IF NOT EXISTS Orders (
38          OrderID INTEGER PRIMARY KEY AUTOINCREMENT,
39          OrderValue REAL,
40          DatePlaced TEXT,
41          DateArriving TEXT,
42          CustomerID INTEGER,
43          VehicleID INTEGER,
44          FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID) ON DELETE CASCADE,
45          FOREIGN KEY (VehicleID) REFERENCES VehicleStock(VehicleID) ON DELETE SET NULL
46      )''')
47
48      # Staff
49      cursor.execute('''
50      CREATE TABLE IF NOT EXISTS Staff (
51          StaffID INTEGER PRIMARY KEY AUTOINCREMENT,
52          FirstName TEXT,
```

```
        LastName TEXT,
        DateOfBirth TEXT,
        PhoneNumber TEXT,
        Position TEXT,
        Title TEXT
    )''')

    # TestDrives
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS TestDrives (
        TestDriveID INTEGER PRIMARY KEY AUTOINCREMENT,
        TestDriveDate TEXT,
        TestDriveTime TEXT,
        Duration TEXT,
        Status TEXT,
        Feedback TEXT,
        CustomerID INTEGER,
        VehicleID INTEGER,
        FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID) ON DELETE CASCADE,
        FOREIGN KEY (VehicleID) REFERENCES VehicleStock(VehicleID) ON DELETE SET NULL
    )''')

    # Access
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS Access (
        AccessID INTEGER PRIMARY KEY AUTOINCREMENT,
        StaffID INTEGER,
        Username TEXT,
        Password TEXT,
        AccessLevel TEXT,
        FOREIGN KEY (StaffID) REFERENCES Staff(StaffID) ON DELETE CASCADE
    )''')

    # Payslips
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS Payslips (
        PayslipID INTEGER PRIMARY KEY AUTOINCREMENT,
        StaffID INTEGER,
        HoursWorked REAL,
        HourlyRate REAL,
        TotalOwed REAL,
        FOREIGN KEY (StaffID) REFERENCES Staff(StaffID) ON DELETE CASCADE
    )''')

    conn.commit()
    conn.close()

```

HelpMenu.py

The helpMenu not only is responsible for the ‘user guide’ but also defines the functions for example that create backups and generate reports.

HELP MENU

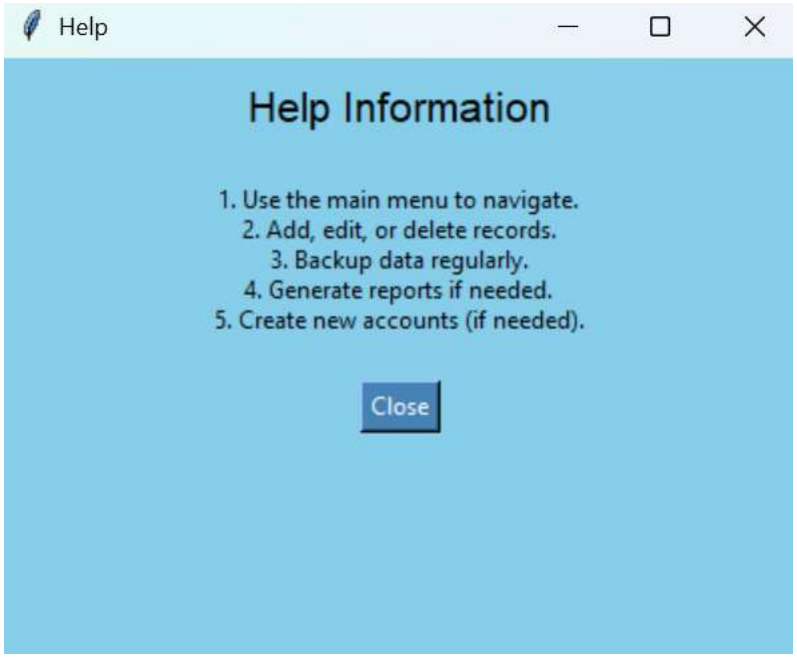
```python
# helpmenu.py

import sqlite3
from tkinter import Toplevel, Label, Button, messagebox
from tkinter.filedialog import asksaveasfilename

def help_page():
    help_window = Toplevel()
    help_window.title("Help")
    help_window.geometry("400x300")
    help_window.config(bg="#87CEEB")

    Label(help_window, text="Help Information", font=("Arial", 16),
          bg="#87CEEB").pack(pady=10)
    Label(help_window,
          text="1. Use the main menu to navigate.\n"
               "2. Add, edit, or delete records.\n"
               "3. Backup data regularly.\n"
               "4. Generate reports if needed.\n"
               "5. Create new accounts (if needed).",
          bg="#87CEEB", wraplength=350).pack(pady=10)
    Button(help_window, text="Close", command=help_window.destroy,
           bg="#468284", fg="white").pack(pady=10)

```



This help menu provides a basic overview of the program with tips to get the most value out of it. For example, reminding users to back up regularly.

BACKUP

```python
def backup_database():
    file_path = asksaveasfilename(defaultextension=".db", filetypes=[("SQLite Database", "*.db")])
    if file_path:
        with open('mulligan_motors.db', 'rb') as original:
            with open(file_path, 'wb') as backup:
                backup.write(original.read())
        messagebox.showinfo("Backup Successful", f"Database backed up to {file_path}")

```



Upon pressing the button, the windows file explorer will open and then the user can input a name and backup location, such as a usb drive.

EXPORT A REPORT

```python
def generate_report(parent_window):
    def do_export():
        chosen_table = table_var.get()
        if not chosen_table:
            messagebox.showwarning("Invalid Selection", "Please select a table.")
            return

        file_path = asksaveasfilename(defaultextension=".txt", filetypes=[("Text Files", "*.txt")])
        if not file_path:
            return

        try:
            conn = sqlite3.connect('mulligan_motors.db')
            cursor = conn.cursor()
            cursor.execute(f"SELECT * FROM {chosen_table}")
            rows = cursor.fetchall()
            conn.close()

            with open(file_path, 'w', encoding='utf-8') as f:
                f.write(f"--- Report for table: {chosen_table} ---\n")
                for row in rows:
                    row_str = ", ".join(str(item) for item in row)
                    f.write(row_str + "\n")

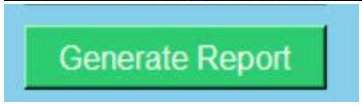
            messagebox.showinfo("Success", f"Report generated: {file_path}")
        except Exception as e:
            messagebox.showerror("Error", f"Could not generate report: {e}")

        report_win.destroy()

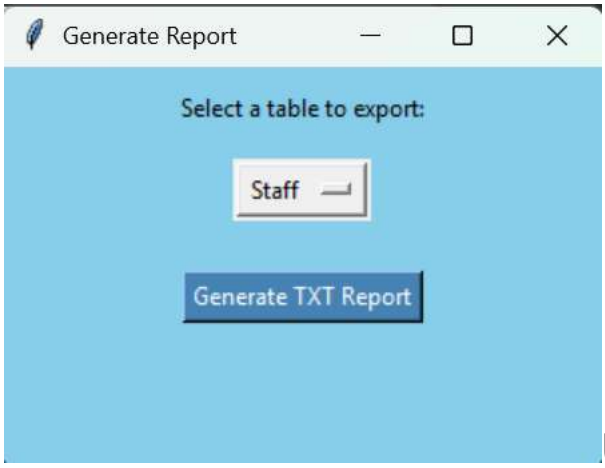
    report_win = Toplevel(parent_window)
    report_win.title("Generate Report")
    report_win.geometry("300x200")
    report_win.config(bg="#87CEEB")

    Label(report_win, text="Select a table to export:", bg="#87CEEB").pack(pady=10)

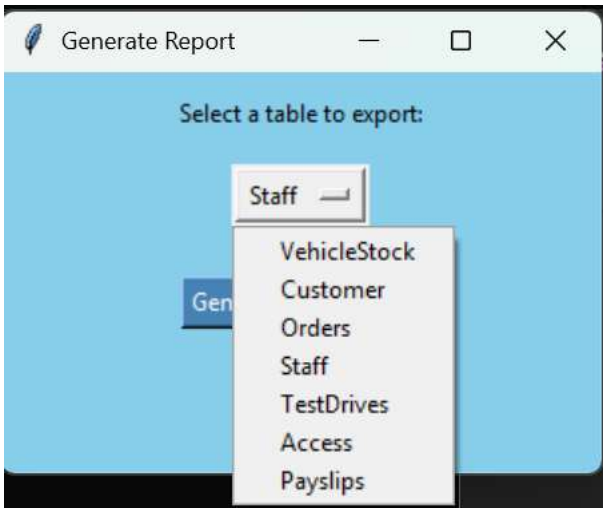
    table_var = StringVar(value="")
    tables_list = [
        "VehicleStock",
        "Customer",
        "Orders",
        "Staff",
        "TestDrives",
        "Access",
        "Payslips"
    ]
    OptionMenu(report_win, table_var, *tables_list).pack(pady=5)
    Button(report_win, text="Generate TXT Report", command=do_export,
           bg="#468284", fg="white").pack(pady=20)
```

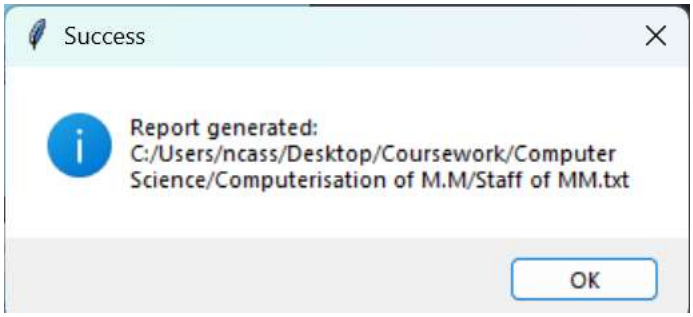


Pressing this button will activate the do_export() function

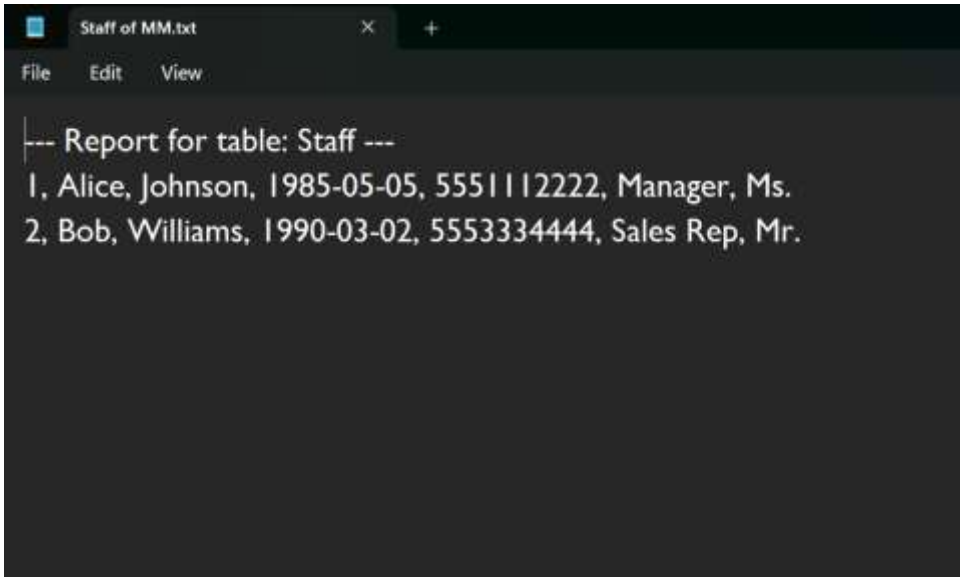


Using a dropdown box, the user selects the table they wish to export.





A notification will occur informing the user of success/failure of the operation



The format of the text file which the program generated report has is seen here.

InputValidation.py

This module contains all the relevant and appropriate validation checks for each table.

```python
# InputValidation.py

import re

def validate_input(table_name, data_dict):
    """
    Basic checks for presence, numeric data, email, etc.
    Returns (True, "") if valid, else (False, errorMsg).
    """

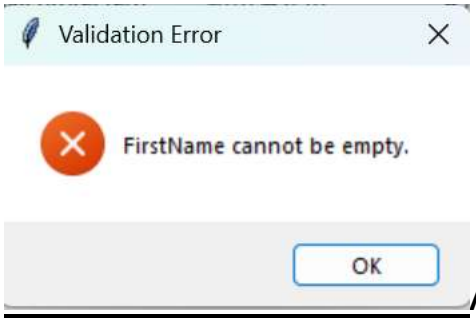
    def is_float(x):
        try:
            float(x)
            return True
        except ValueError:
            return False

    def is_int(x):
        try:
            int(x)
            return True
        except ValueError:
            return False

    email_pattern = r"^[\w\.-]+@[\w\.-]+\.\w+$"

    #presence checks
    if table_name == "VehicleStock":
        if not data_dict.get("VehicleMake", "").strip():
            return False, "VehicleMake cannot be empty."
    elif table_name == "Customer":
        if not data_dict.get("FirstName", "").strip():
            return False, "FirstName cannot be empty."
    elif table_name == "Orders":
        if not data_dict.get("DatePlaced", "").strip():
            return False, "DatePlaced cannot be empty."
    elif table_name == "Staff":
        if not data_dict.get("FirstName", "").strip():
            return False, "FirstName cannot be empty."
    elif table_name == "TestDrives":
        if not data_dict.get("TestDriveDate", "").strip():
            return False, "TestDriveDate cannot be empty."
    elif table_name == "Access":
        if not data_dict.get("Username", "").strip():
            return False, "Username cannot be empty."
    elif table_name == "Payslips":
        if not data_dict.get("StaffID", "").strip():
            return False, "StaffID cannot be empty."
```

```
49
50      # numeric checks
51      if table_name == "VehicleStock":
52          if data_dict.get("VehiclePrice") and not is_float(data_dict["VehiclePrice"]):
53              return False, "VehiclePrice must be numeric."
54          if data_dict.get("ItemQuantity") and not is_int(data_dict["ItemQuantity"]):
55              return False, "ItemQuantity must be integer."
56
57      # ... more checks for Customer, Orders, etc. ...
58
59      return True, ""
60
```



An example of a validation error occurring can be seen above.

Insert DB Code for MM

This module includes example data that can be inserted into the solution for testing.

```python
import sqlite3

def populate_database():
    conn = sqlite3.connect('mulligan_motors.db')
    cursor = conn.cursor()

    # ------------------------------
    # 1. Insert sample VehicleStock
    # ------------------------------
    vehicle_data = [
        ("Toyota", "Corolla", "Sedan", 20000, 5, "Slightly Used"),
        ("Honda", "Civic", "Sedan", 18000, 3, "Pristine"),
        ("Ford", "F-150", "Truck", 35000, 2, "Heavily Used"),
    ]
    cursor.executemany('''
        INSERT INTO VehicleStock (VehicleMake, VehicleName, VehicleType, VehiclePrice, ItemQuantity, VehicleCondition)
        VALUES (?, ?, ?, ?, ?, ?)
    ''', vehicle_data)

    # ------------------------------
    # 2. Insert sample Customer
    # ------------------------------
    customer_data = [
        ("John", "Doe", "[email protected]", "1234567890", "123 Elm Street", "Phone", "2025-01-10"),
        ("Jane", "Smith", "[email protected]", "9876543210", "456 Oak Avenue", "Email", "2025-01-15"),
    ]
    cursor.executemany('''
        INSERT INTO Customer (FirstName, LastName, Email, PhoneNumber, Address, PreferredContactMethod, DateJoined)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', customer_data)

    # ------------------------------------
    # 3. Insert sample Orders (FKs needed)
    # ------------------------------------
    # Assuming VehicleStock auto-increment IDs start at 1,2,3...
    # Assuming Customer auto-increment IDs start at 1,2...
    order_data = [
        (20000, "2025-01-12", "2025-01-20", 1, 1),  # CustomerID=1, VehicleID=1
        (18000, "2025-01-14", "2025-01-25", 2, 2),  # CustomerID=2, VehicleID=2
    ]
    cursor.executemany('''
        INSERT INTO Orders (OrderValue, DatePlaced, DateArriving, CustomerID, VehicleID)
        VALUES (?, ?, ?, ?, ?)
    ''', order_data)

    # ------------------------------
    # 4. Insert sample Staff
    # ------------------------------
```

```
    staff_data = [
        ("Alice", "Johnson", "1985-05-05", "5551112222", "Manager", "Ms."),
        ("Bob",   "Williams", "1990-03-02", "5553334444", "Sales Rep", "Mr."),
    ]
    cursor.executemany('''
        INSERT INTO Staff (FirstName, LastName, DateOfBirth, PhoneNumber, Position, Title)
        VALUES (?, ?, ?, ?, ?, ?)
    ''', staff_data)

    # ----------------------------
    # 5. Insert sample Access (FK)
    # ----------------------------
    # Corresponding to the Staff above (StaffID=1 => admin, StaffID=2 => sales)
    access_data = [
        (1, "alice_admin", "adminPass", "admin"),
        (2, "bob_sales", "salesPass", "sales"),
    ]
    cursor.executemany('''
        INSERT INTO Access (StaffID, Username, Password, AccessLevel)
        VALUES (?, ?, ?, ?)
    ''', access_data)

    # ----------------------------------------
    # 6. Insert sample TestDrives (FK CustomerID/VehicleID)
    # ----------------------------------------
    testdrive_data = [
        ("2025-02-01", "10:00", "30 minutes", "Completed", "Loved it!", 1, 2),
        ("2025-02-05", "14:00", "45 minutes", "Pending", "", 2, 3),
    ]
    cursor.executemany('''
        INSERT INTO TestDrives (TestDriveDate, TestDriveTime, Duration, Status, Feedback, CustomerID, VehicleID)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', testdrive_data)

    # ----------------------------------------
    # 7. Insert sample Payslips (FK StaffID)
    # ----------------------------------------
    payslips_data = [
        (1, 40, 20.0, 40 * 20.0),  # StaffID=1 => total owed = hours * rate
        (2, 35, 15.0, 35 * 15.0),  # StaffID=2
    ]
    cursor.executemany('''
        INSERT INTO Payslips (StaffID, HoursWorked, HourlyRate, TotalOwed)
        VALUES (?, ?, ?, ?)
    ''', payslips_data)
```

```
    # -------------
    # Finalize
    # -------------
    conn.commit()
    conn.close()
    print("Sample data inserted successfully!")

if __name__ == "__main__":
    populate_database()
```

LoginPage

This module defines the user login flow for Mulligan Motors, such as the ability to create new accounts. When the user runs 'login_screen()', they see a window to input username and password. The system checks the 'Access' table for a matching record and launches either manager_home

(if admin) or sales_home (if sales). It also provides 'create_account_window()' so new accounts (StaffID, Username, Password, AccessLevel) can be added to the Access table.

Creating an account

```
    def register_account():
        staff_id_str = staff_id_var.get()
        username = username_var.get().strip()
        pw = password_var.get()
        pw_confirm = confirm_var.get()
        access_level = access_var.get()

        if not staff_id_str or not username or not pw or not pw_confirm:
            messagebox.showerror("Validation Error", "All fields are required.")
            return

        try:
            staff_id_int = int(staff_id_str)
        except ValueError:
            messagebox.showerror("Validation Error", "StaffID must be a number.")
            return

        if pw != pw_confirm:
            messagebox.showerror("Validation Error", "Passwords do not match.")
            return
```

```
        try:
            conn = sqlite3.connect('mulligan_motors.db')
            cursor = conn.cursor()
            cursor.execute("SELECT AccessID FROM Access WHERE Username = ?", (username,))
            existing = cursor.fetchone()
            if existing:
                messagebox.showerror("Error", f"Username '{username}' is already taken.")
                conn.close()
                return

            cursor.execute('''
                INSERT INTO Access (StaffID, Username, Password, AccessLevel)
                VALUES (?, ?, ?, ?)
            ''', (staff_id_int, username, pw, access_level))
            conn.commit()
            conn.close()
            messagebox.showinfo("Success", f"Account '{username}' created successfully!")
            create_win.destroy()
        except Exception as e:
            messagebox.showerror("Database Error", str(e))

    create_win = Toplevel(parent_window)
    create_win.title("Create Account")
    create_win.geometry("400x400")
    create_win.config(bg="#87CEEB")

    Label(create_win, text="Enter details to create an account", font=("Arial", 14),
          bg="#87CEEB").pack(pady=10)
```

```
    staff_id_var = StringVar()
    username_var = StringVar()
    password_var = StringVar()
    confirm_var = StringVar()
    access_var = StringVar(value="sales")

    Label(create_win, text="Staff ID:", bg="#87CEEB").pack(pady=5)
    Entry(create_win, textvariable=staff_id_var).pack(pady=5)

    Label(create_win, text="Username:", bg="#87CEEB").pack(pady=5)
    Entry(create_win, textvariable=username_var).pack(pady=5)

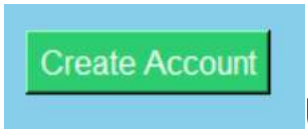
    Label(create_win, text="Password:", bg="#87CEEB").pack(pady=5)
    Entry(create_win, textvariable=password_var, show="*").pack(pady=5)

    Label(create_win, text="Confirm Password:", bg="#87CEEB").pack(pady=5)
    Entry(create_win, textvariable=confirm_var, show="*").pack(pady=5)

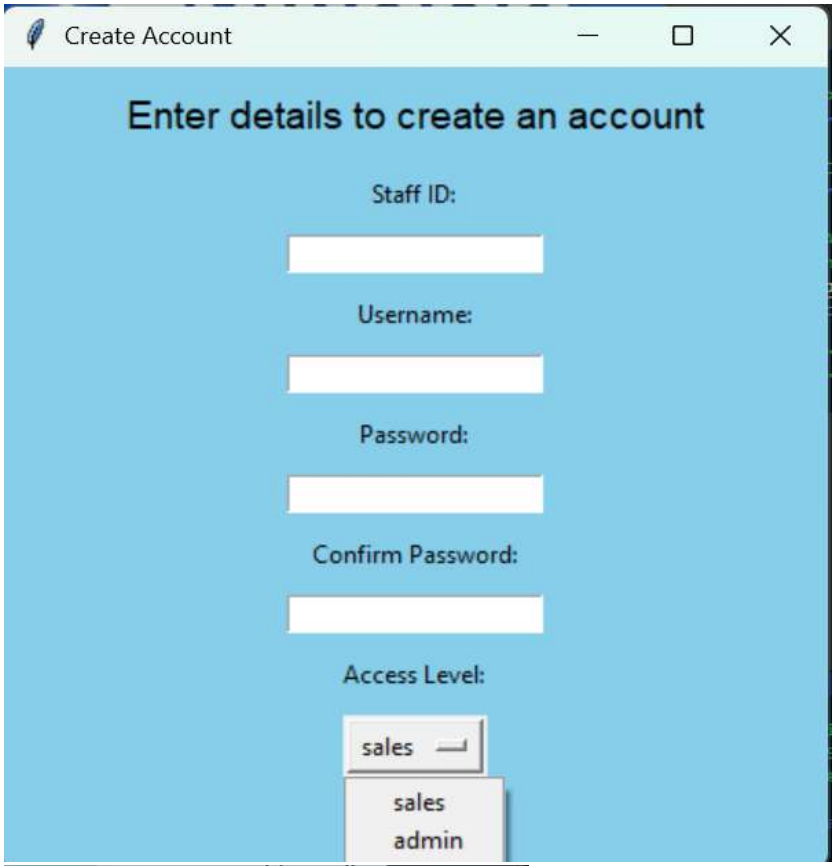
    Label(create_win, text="Access Level:", bg="#87CEEB").pack(pady=5)
    levels = ["sales", "admin"]
    from tkinter import OptionMenu
    OptionMenu(create_win, access_var, *levels).pack(pady=5)

    Button(create_win, text="Register", command=register_account,
           bg="#4682B4", fg="white").pack(pady=20)

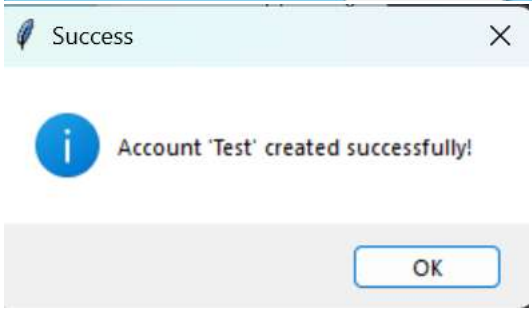
```



Upon pressing this button, the account creation process with a empty form opening.



The user can then fill in the account details of the account they wish to create. If the password and username are already in use, the program will prevent the creation of a new account.



If all details are valid, a new account will

Main Login page

```
85  def login_screen():
86      login_window = Tk()
87      login_window.title("Mulligan Motors - Login")
88      login_window.geometry("450x350")
89      login_window.config(bg="#87CEEB")
90
91      # Attempt to load mulligan_logo1.png with PhotoImage
92      try:
93          logo1_img = PhotoImage(file="mulligan_logo1.png")
94          logo_label = Label(login_window, image=logo1_img, bg="#87CEEB")
95          logo_label.image = logo1_img  # keep a reference
96          logo_label.pack(pady=10)
97      except Exception as e:
98          print("Could not load mulligan_logo1.png:", e)
99
100     Label(login_window, text="Login to Mulligan Motors", font=("Arial", 16),
101           bg="#87CEEB", fg="black").pack(pady=5)
102
103     username_var = StringVar()
104     password_var = StringVar()
105
106     Label(login_window, text="Username", font=("Arial", 12), bg="#87CEEB").pack(pady=5)
107     Entry(login_window, textvariable=username_var, font=("Arial", 12)).pack(pady=5)
108
109     Label(login_window, text="Password", font=("Arial", 12), bg="#87CEEB").pack(pady=5)
110     Entry(login_window, textvariable=password_var, show="*", font=("Arial", 12)).pack(pady=5)
```



Login Verification

```
112     def validate_login():
113         user = username_var.get()
114         pw = password_var.get()
115         conn = sqlite3.connect('mulligan_motors.db')
116         cursor = conn.cursor()
117         cursor.execute('''
118             SELECT AccessLevel
119             FROM Access
120             WHERE Username = ? AND Password = ?
121         ''', (user, pw))
122         row = cursor.fetchone()
123         conn.close()
124 
125         if row is None:
126             messagebox.showerror("Login Failed", "Invalid username or password.")
127         else:
128             level = row[0].lower()
129             login_window.destroy()
130             if level == "admin":
131                 manager_home()
132             elif level == "sales":
133                 sales_home()
134             else:
135                 messagebox.showinfo("Access Restricted",
136                                     f"Unrecognized AccessLevel: {level}")
137 
138     Button(login_window, text="Login", command=validate_login,
139            font=("Arial", 12), bg="#4682B4", fg="white").pack(pady=10)
140 
141     Button(login_window, text="Create Account",
142            font=("Arial", 12), bg="#2ecc71", fg="white",
143            command=lambda: create_account_window(login_window)).pack(pady=5)
144 
145     login_window.mainloop()
146
```



Manager.py

Window & logo

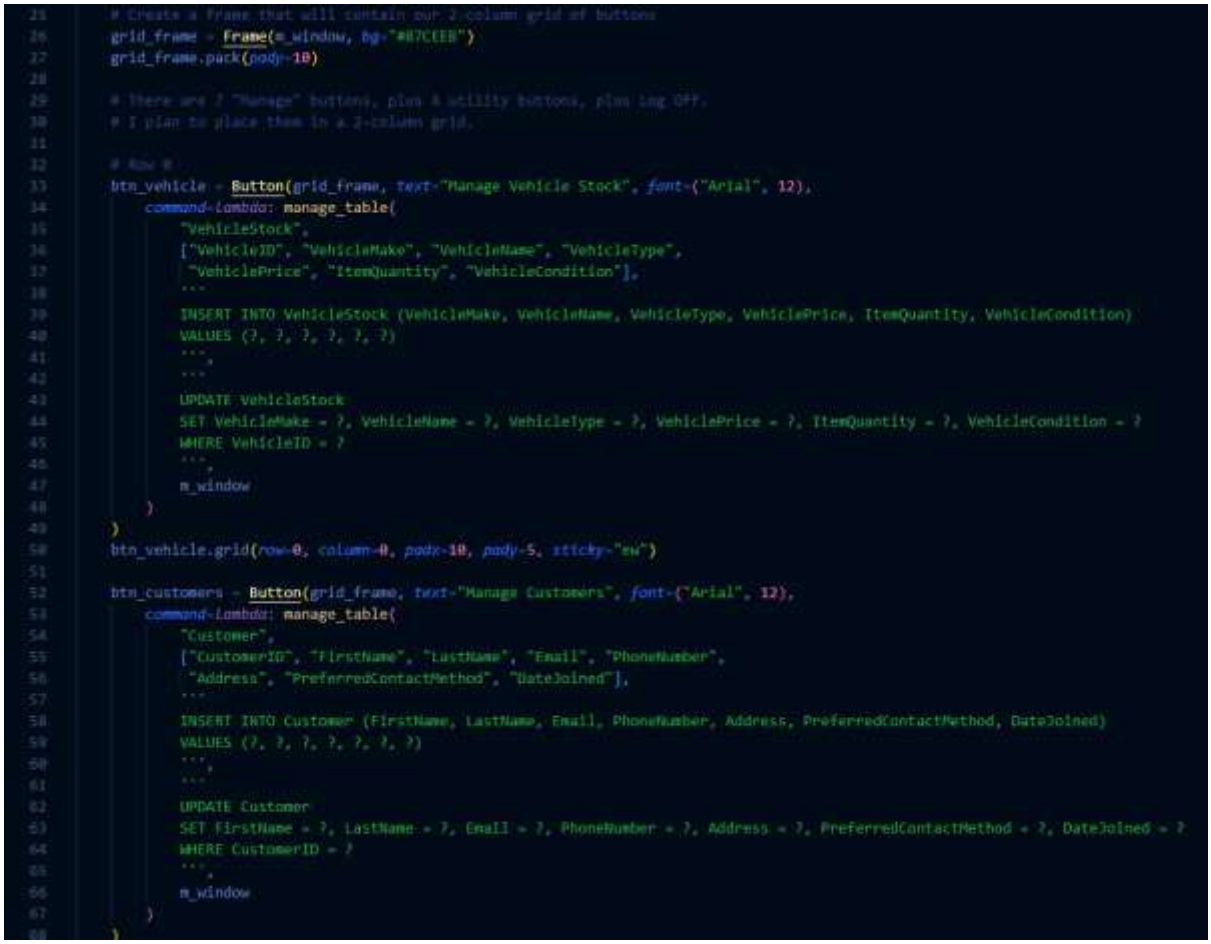
```python
def manager_home():
    m_window = Tk()
    m_window.title("Mulligan Motors - Manager")
    m_window.geometry("600x600")  # Set window size
    m_window.config(bg="#87CEEB")

    # Attempt to load the second Mulligan Motors image
    try:
        logo2_img = PhotoImage(file="mulligan_logo2.png")
        logo2_label = Label(m_window, image=logo2_img, bg="#87CEEB")
        logo2_label.image = logo2_img
        logo2_label.pack(pady=10)  # pack for the logo
    except Exception as e:
        print("Could not load mulligan_logo2.png:", e)

    Label(m_window, text="Mulligan Motors Management (ADMIN)",
          font=("Arial", 16), bg="#87CEEB", fg="black").pack(pady=10)
```



Adding in buttons

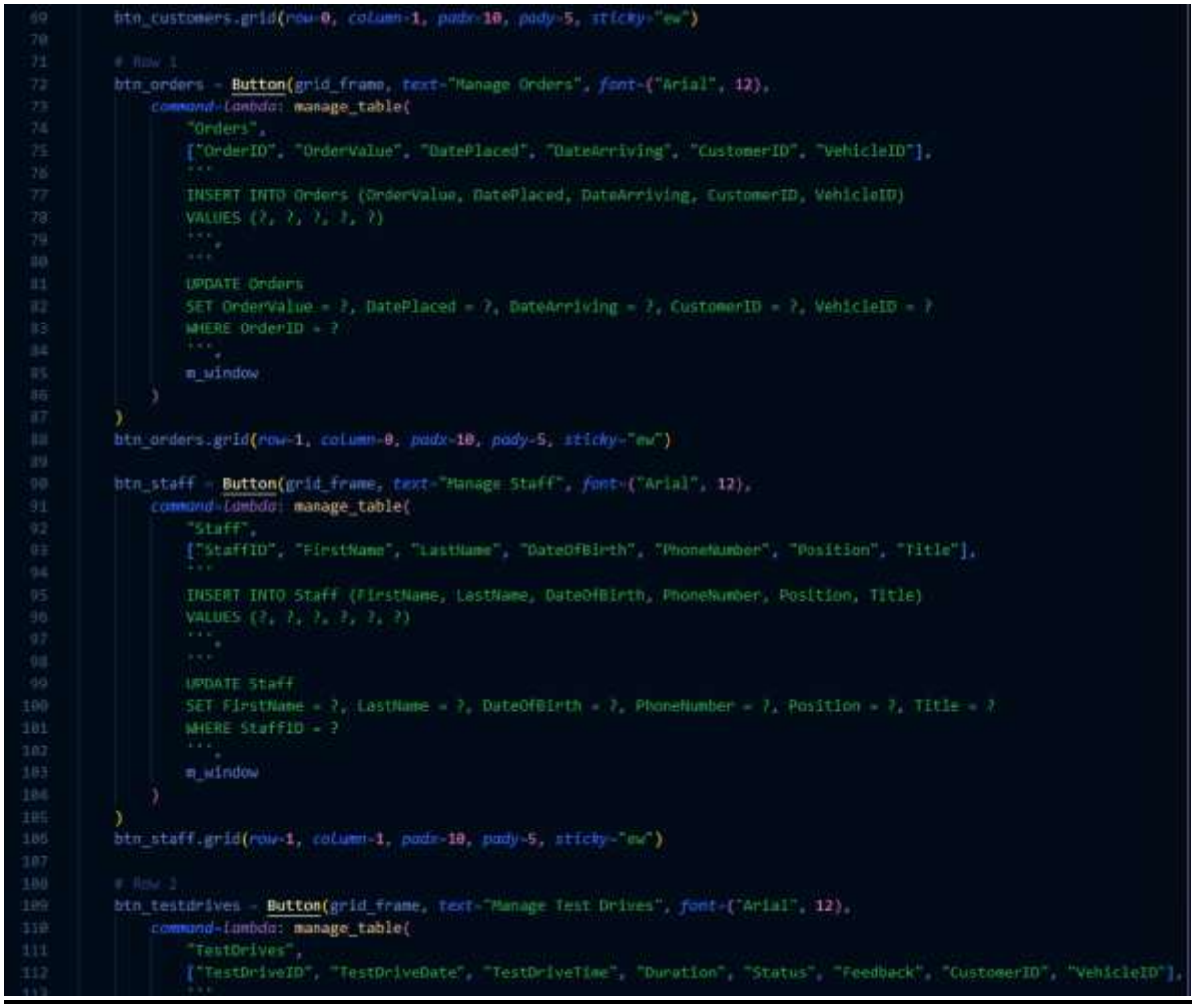


```
    # Create a Frame that will contain our 2-column grid of buttons
    grid_frame = Frame(m_window, bg="#87CEEB")
    grid_frame.pack(pady=10)

    # There are 7 "Manage" buttons, plus 4 utility buttons, plus Log Off.
    # I plan to place them in a 2-column grid.

    # Row 0
    btn_vehicle = Button(grid_frame, text="Manage Vehicle Stock", font=("Arial", 12),
        command=lambda: manage_table(
            "VehicleStock",
            ["VehicleID", "VehicleMake", "VehicleName", "VehicleType",
             "VehiclePrice", "ItemQuantity", "VehicleCondition"],
            """
            INSERT INTO VehicleStock (VehicleMake, VehicleName, VehicleType, VehiclePrice, ItemQuantity, VehicleCondition)
            VALUES (?, ?, ?, ?, ?, ?)
            """,
            """
            UPDATE VehicleStock
            SET VehicleMake = ?, VehicleName = ?, VehicleType = ?, VehiclePrice = ?, ItemQuantity = ?, VehicleCondition = ?
            WHERE VehicleID = ?
            """,
            m_window
        )
    )
    btn_vehicle.grid(row=0, column=0, padx=10, pady=5, sticky="ew")

    btn_customers = Button(grid_frame, text="Manage Customers", font=("Arial", 12),
        command=lambda: manage_table(
            "Customer",
            ["CustomerID", "FirstName", "LastName", "Email", "PhoneNumber",
             "Address", "PreferredContactMethod", "DateJoined"],
            """
            INSERT INTO Customer (FirstName, LastName, Email, PhoneNumber, Address, PreferredContactMethod, DateJoined)
            VALUES (?, ?, ?, ?, ?, ?, ?)
            """,
            """
            UPDATE Customer
            SET FirstName = ?, LastName = ?, Email = ?, PhoneNumber = ?, Address = ?, PreferredContactMethod = ?, DateJoined = ?
            WHERE CustomerID = ?
            """,
            m_window
        )
    )
```



```
    btn_customers.grid(row=0, column=1, padx=10, pady=5, sticky="ew")

    # Row 1
    btn_orders = Button(grid_frame, text="Manage Orders", font=("Arial", 12),
        command=lambda: manage_table(
            "Orders",
            ["OrderID", "OrderValue", "DatePlaced", "DateArriving", "CustomerID", "VehicleID"],
            '''
            INSERT INTO Orders (OrderValue, DatePlaced, DateArriving, CustomerID, VehicleID)
            VALUES (?, ?, ?, ?, ?)
            ''',
            '''
            UPDATE Orders
            SET OrderValue = ?, DatePlaced = ?, DateArriving = ?, CustomerID = ?, VehicleID = ?
            WHERE OrderID = ?
            ''',
            m_window
        )
    )
    btn_orders.grid(row=1, column=0, padx=10, pady=5, sticky="ew")

    btn_staff = Button(grid_frame, text="Manage Staff", font=("Arial", 12),
        command=lambda: manage_table(
            "Staff",
            ["StaffID", "FirstName", "LastName", "DateOfBirth", "PhoneNumber", "Position", "Title"],
            '''
            INSERT INTO Staff (FirstName, LastName, DateOfBirth, PhoneNumber, Position, Title)
            VALUES (?, ?, ?, ?, ?, ?)
            ''',
            '''
            UPDATE Staff
            SET FirstName = ?, LastName = ?, DateOfBirth = ?, PhoneNumber = ?, Position = ?, Title = ?
            WHERE StaffID = ?
            ''',
            m_window
        )
    )
    btn_staff.grid(row=1, column=1, padx=10, pady=5, sticky="ew")

    # Row 2
    btn_testdrives = Button(grid_frame, text="Manage Test Drives", font=("Arial", 12),
        command=lambda: manage_table(
            "TestDrives",
            ["TestDriveID", "TestDriveDate", "TestDriveTime", "Duration", "Status", "Feedback", "CustomerID", "VehicleID"],
            '''
```

```
            INSERT INTO TestDrives (TestDriveDate, TestDriveTime, Duration, Status, Feedback, CustomerID, VehicleID)
            VALUES (?, ?, ?, ?, ?, ?, ?)
            ''',
            '''
            UPDATE TestDrives
            SET TestDriveDate = ?, TestDriveTime = ?, Duration = ?, Status = ?, Feedback = ?, CustomerID = ?, VehicleID = ?
            WHERE TestDriveID = ?
            ''',
            m_window
        )
    )
    btn_testdrives.grid(row=2, column=0, padx=10, pady=5, sticky="ew")

    btn_access = Button(grid_frame, text="Manage Access", font=("Arial", 12),
        command=lambda: manage_table(
            "Access",
            ["AccessID", "StaffID", "Username", "Password", "AccessLevel"],
            '''
            INSERT INTO Access (StaffID, Username, Password, AccessLevel)
            VALUES (?, ?, ?, ?)
            ''',
            '''
            UPDATE Access
            SET StaffID = ?, Username = ?, Password = ?, AccessLevel = ?
            WHERE AccessID = ?
            ''',
            m_window
        )
    )
    btn_access.grid(row=2, column=1, padx=10, pady=5, sticky="ew")

    # Row 3
    btn_payslips = Button(grid_frame, text="Manage Payslips", font=("Arial", 12),
        command=lambda: manage_table(
            "Payslips",
            ["PayslipID", "StaffID", "HoursWorked", "HourlyRate", "TotalOwed"],
            '''
            INSERT INTO Payslips (StaffID, HoursWorked, HourlyRate, TotalOwed)
            VALUES (?, ?, ?, ?)
            ''',
            '''
            UPDATE Payslips
            SET StaffID = ?, HoursWorked = ?, HourlyRate = ?, TotalOwed = ?
            WHERE PayslipID = ?
            ''',
```

```
            ''',
            m_window
        )
    )
    btn_payslips.grid(row=3, column=0, padx=10, pady=5, sticky="ew")
```



Utility Buttons

```
    # Utility Buttons
    # I will put them on row 3 or 4, column 1 or 0
    btn_report = Button(grid_frame, text="Generate Report", font=("Arial", 12),
        command=lambda: generate_report(m_window),
        bg="#2ecc71", fg="white"
    )
    btn_report.grid(row=3, column=1, padx=10, pady=5, sticky="ew")

    btn_backup = Button(grid_frame, text="Backup Database", font=("Arial", 12),
        command=backup_database, bg="#4682B4", fg="white"
    )
    btn_backup.grid(row=4, column=0, padx=10, pady=5, sticky="ew")

    btn_help = Button(grid_frame, text="Help", font=("Arial", 12),
        command=help_page, bg="#4682B4", fg="white"
    )
    btn_help.grid(row=4, column=1, padx=10, pady=5, sticky="ew")

    # Log Off button on row 5, col 1, to place it bottom-right
    btn_logoff = Button(grid_frame, text="Log Off", font=("Arial", 12),
        command=m_window.destroy, bg="#f39c12", fg="white"
    )
    btn_logoff.grid(row=5, column=1, padx=10, pady=10, sticky="e")

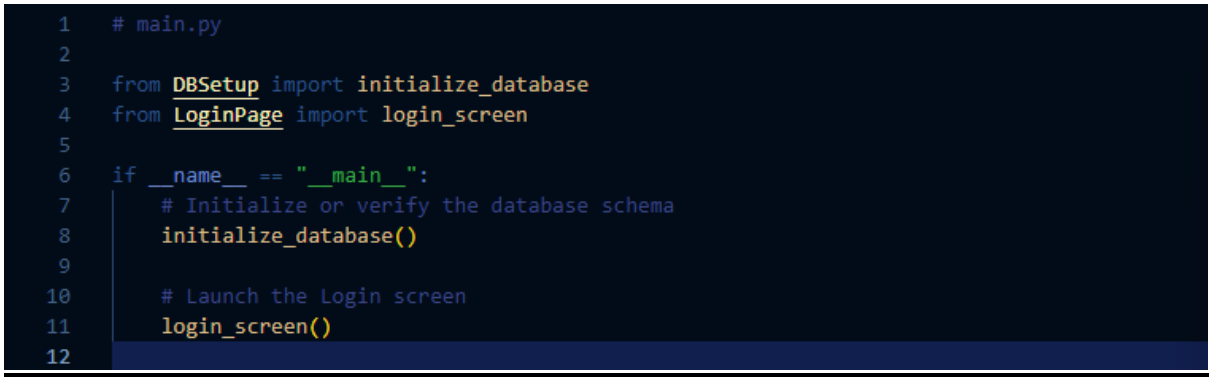
    # Make columns expand a bit if we resize the window
    grid_frame.columnconfigure(0, weight=1)
    grid_frame.columnconfigure(1, weight=1)

    m_window.mainloop()

```



Main.py



```python
# main.py

from DBSetup import initialize_database
from LoginPage import login_screen

if __name__ == "__main__":
    # Initialize or verify the database schema
    initialize_database()

    # Launch the Login screen
    login_screen()

```

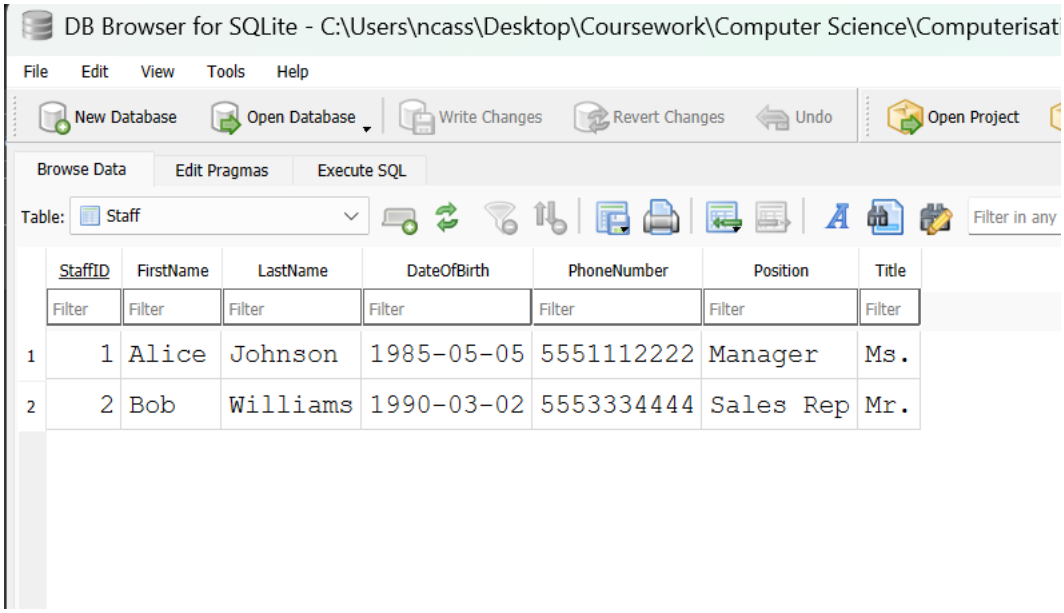
manageTable.py

This files defines the processes used to manipulate data and the database in the program, such as adding, deleting, searching or editing a record. For purposes of efficiency, the edit function calls upon the add() and delete() function which reduces the amount of redundant lines of code in the program.

Retrieving data

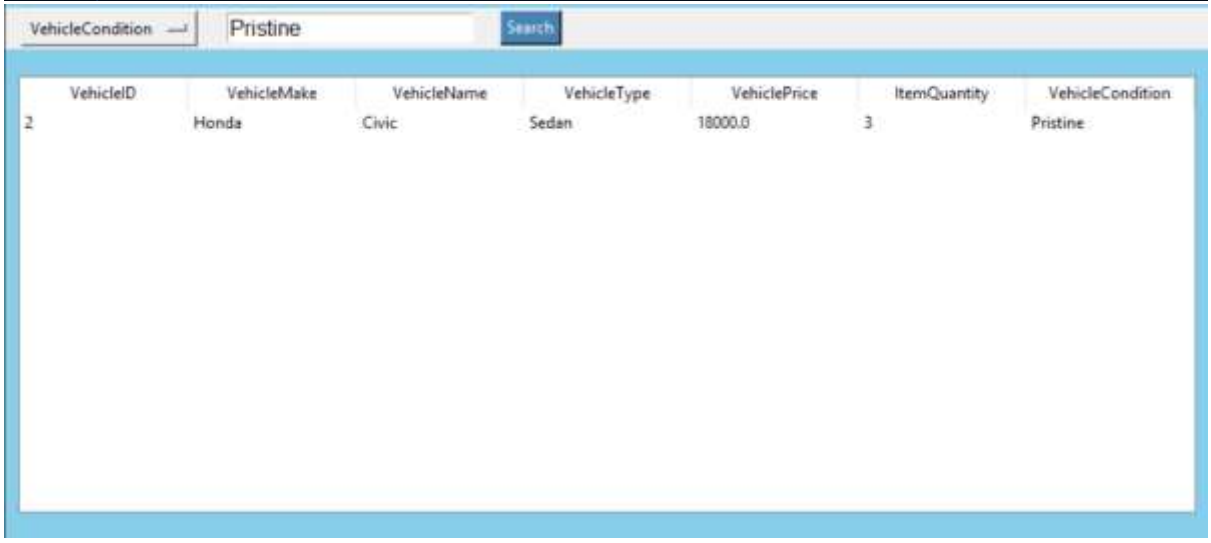
```
11
12      def load_data():
13          for item in table_tree.get_children():
14              table_tree.delete(item)
15          conn = sqlite3.connect('mulligan_motors.db')
16          cursor = conn.cursor()
17          cursor.execute(f"SELECT * FROM {table_name}")
18          rows = cursor.fetchall()
19          conn.close()
20          for row in rows:
21              table_tree.insert("", "end", values=row)
```



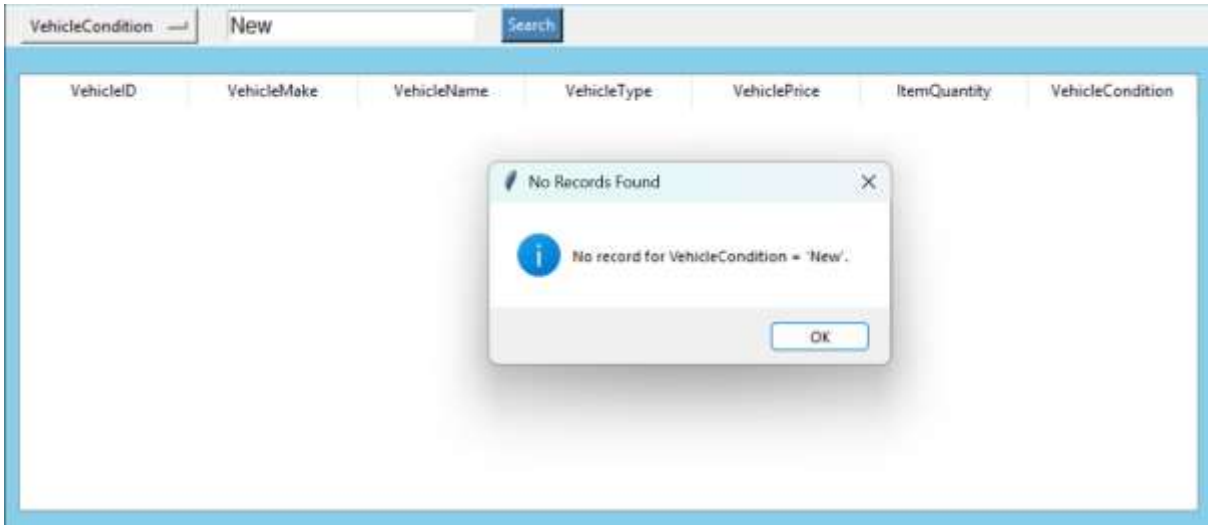


Searching a record

```
23      def search():
24          col = dropdown_var.get()
25          term = search_entry.get()
26          if not term:
27              messagebox.showwarning("Invalid Input", "Please enter a search term.")
28              return
29          conn = sqlite3.connect('mulligan_motors.db')
30          cursor = conn.cursor()
31          query = f"SELECT * FROM {table_name} WHERE {col} LIKE ?"
32          cursor.execute(query, (f"%{term}%",))
33          results = cursor.fetchall()
34          conn.close()
35
36          for item in table_tree.get_children():
37              table_tree.delete(item)
38          if results:
39              for r in results:
40                  table_tree.insert("", "end", values=r)
41          else:
42              messagebox.showinfo("No Records Found", f"No record for {col} = '{term}'.")
```



Searching a record will display **only** the records that match.



If a record cannot be found, this error window is displayed.

Searching User Interface

```
202     # Search area
203     search_frame = ttk.Frame(table_window)
204     search_frame.pack(pady=10, fill="x")
205
206     dropdown_var = StringVar()
207     dropdown_var.set(fields[0])
208     OptionMenu(search_frame, dropdown_var, *fields).pack(side="left", padx=10)
209
210     search_entry = Entry(search_frame, font=("Arial", 12), width=20)
211     search_entry.pack(side="left", padx=10)
212     Button(search_frame, text="Search", command=search,
213            bg="#4682B4", fg="white").pack(side="left", padx=10)
```

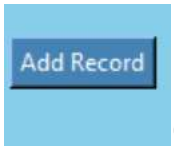


The user has the option of searching for a record using any of the entities' attributes.

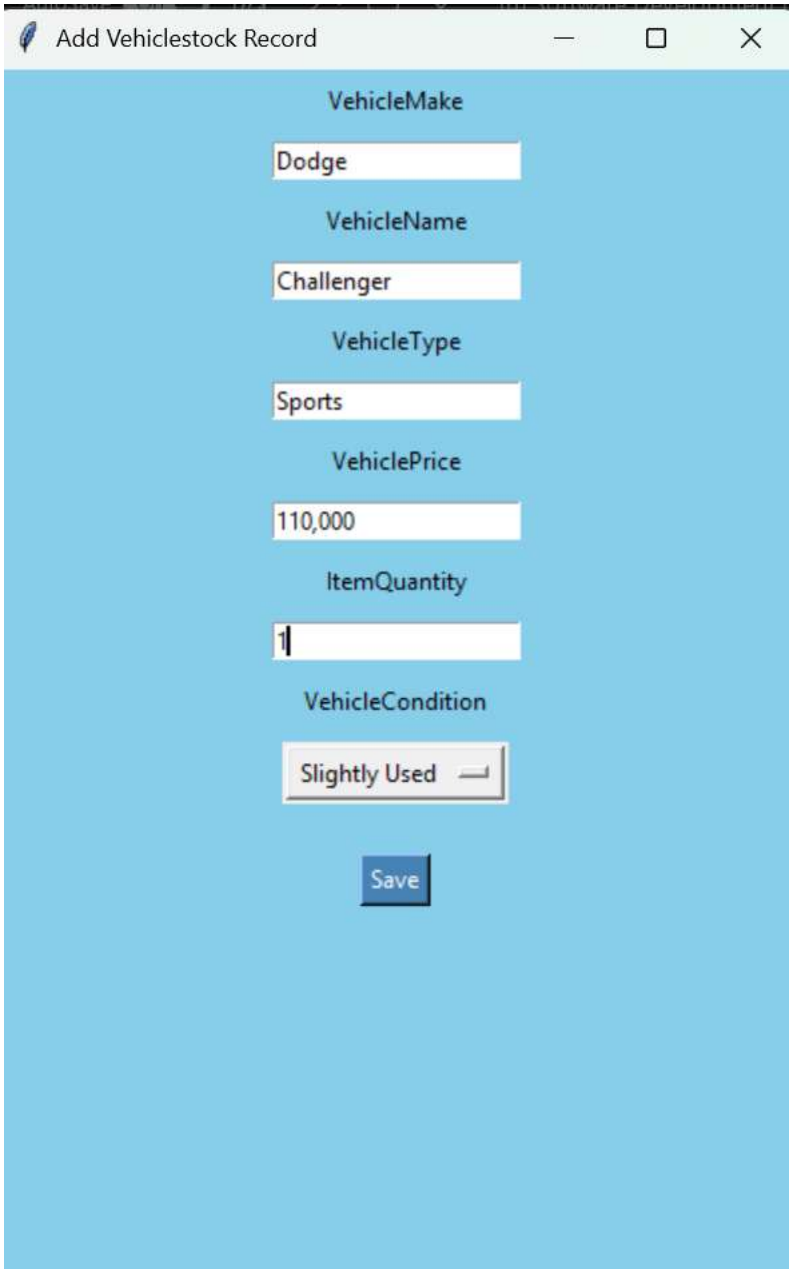
Adding a record

```
44      def add_record():
45          def save_record():
46              add_fields_final = fields[1:]
47              if table_name == "Payslips":
48                  add_fields_final = [x for x in add_fields_final if x != "TotalOwed"]
49
50              record_data = {}
51              for ff in add_fields_final:
52                  record_data[ff] = entry_vars[ff].get()
53
54              is_valid, error_msg = validate_input(table_name, record_data)
55              if not is_valid:
56                  messagebox.showerror("Validation Error", error_msg)
57                  return
58
59              if table_name == "Payslips":
60                  s_id = record_data["StaffID"]
61                  hrs = float(record_data["HoursWorked"])
62                  rate = float(record_data["HourlyRate"])
63                  total = hrs * rate
64                  conn = sqlite3.connect('mulligan_motors.db')
65                  cursor = conn.cursor()
66                  cursor.execute(insert_query, (s_id, hrs, rate, total))
67                  conn.commit()
68                  conn.close()
69              else:
70                  vals = [record_data[ff] for ff in add_fields_final]
71                  conn = sqlite3.connect('mulligan_motors.db')
72                  cursor = conn.cursor()
73                  cursor.execute(insert_query, vals)
74                  conn.commit()
75                  conn.close()
76
77              add_win.destroy()
78              load_data()
79
80          add_win = Toplevel(parent_window)
81          add_win.title(f"Add {table_name.title()} Record")
82          add_win.geometry("400x600")
83          add_win.config(bg="#87CEEB")
84
85          add_fields_list = fields[1:]
```

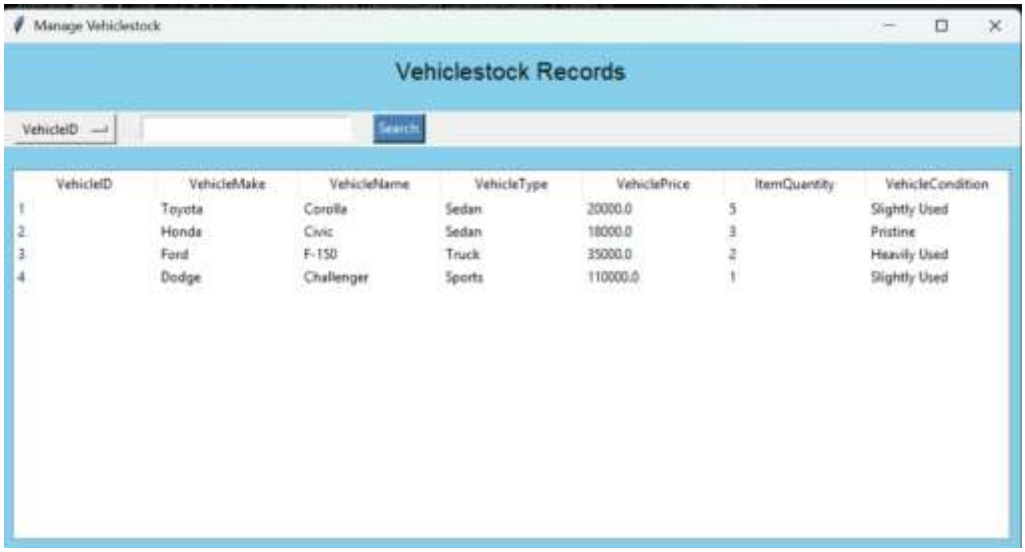
```
86          if table_name == "Payslips":
87              add_fields_list = [x for x in add_fields_list if x != "TotalOwed"]
88
89          entry_vars = {}
90          for ff in add_fields_list:
91              Label(add_win, text=ff, bg="#87CEEB").pack(pady=5)
92
93              # e.g. drop-downs for VehicleCondition, PreferredContactMethod
94              if table_name == "VehicleStock" and ff == "VehicleCondition":
95                  var = StringVar(value="Pristine")
96                  OptionMenu(add_win, var, "Pristine", "Slightly Used", "Heavily Used").pack
97              elif table_name == "Customer" and ff == "PreferredContactMethod":
98                  var = StringVar(value="Email")
99                  OptionMenu(add_win, var, "Email", "Phone", "SMS").pack(pady=5)
100             else:
101                 var = StringVar()
102                 Entry(add_win, textvariable=var).pack(pady=5)
103             entry_vars[ff] = var
104
105         Button(add_win, text="Save", command=save_record,
106                bg="#4682B4", fg="white").pack(pady=20)
```



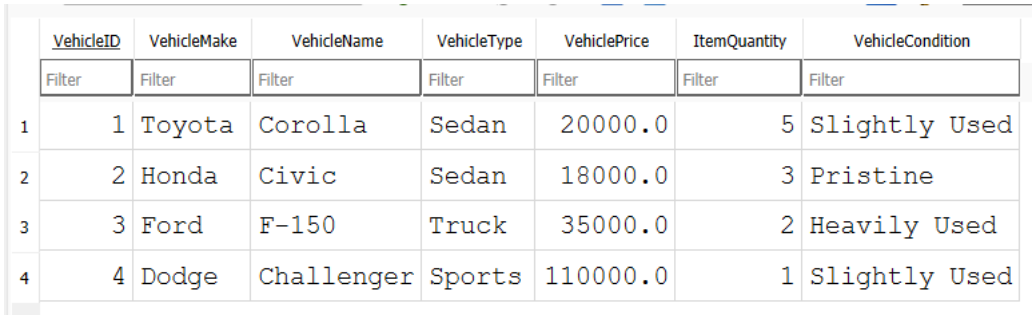
Clicking this button will start the record adding process.



On a separate window, an empty template will pop up. The user can then fill it out (as seen in the screenshot) and can press save to enter it.



As we can see, the record has been added to the treeview viewer.



	VehicleID	VehicleMake	VehicleName	VehicleType	VehiclePrice	ItemQuantity	VehicleCondition
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Toyota	Corolla	Sedan	20000.0	5	Slightly Used
2	2	Honda	Civic	Sedan	18000.0	3	Pristine
3	3	Ford	F-150	Truck	35000.0	2	Heavily Used
4	4	Dodge	Challenger	Sports	110000.0	1	Slightly Used

Furthermore, this new record has been permanently added to the database's storage.

Editing a record

```
108     def edit_record():
109         sel = table_tree.selection()
110         if not sel:
111             messagebox.showerror("No Selection", "Select a record to edit.")
112             return
113         old_vals = table_tree.item(sel)["values"]
114 
115         def save_edits():
116             pk = old_vals[0]
117             record_data = {}
118             for col_name in edit_fields_list:
119                 record_data[col_name] = entry_vars[col_name].get()
120 
121             is_valid, error_msg = validate_input(table_name, record_data)
122             if not is_valid:
123                 messagebox.showerror("Validation Error", error_msg)
124                 return
125 
126             if table_name == "Payslips":
127                 st_id = record_data["StaffID"]
128                 hrs2 = float(record_data["HoursWorked"])
129                 rate2 = float(record_data["HourlyRate"])
130                 tot2 = hrs2 * rate2
131                 conn = sqlite3.connect('mulligan_motors.db')
132                 cursor = conn.cursor()
133                 cursor.execute(update_query, (st_id, hrs2, rate2, tot2, pk))
134                 conn.commit()
135                 conn.close()
136             else:
137                 new_vals = [record_data[f2] for f2 in edit_fields_list]
138                 new_vals.append(pk)
139                 conn = sqlite3.connect('mulligan_motors.db')
140                 cursor = conn.cursor()
141                 cursor.execute(update_query, new_vals)
142                 conn.commit()
143                 conn.close()
144 
145             edit_win.destroy()
146             load_data()
147 
148         edit_win = Toplevel(parent_window)
149         edit_win.title(f"Edit {table_name.title()} Record")
```

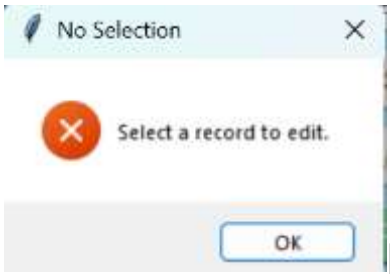
```
150         edit_win.geometry("400x600")
151         edit_win.config(bg="#87CEEB")
152
153         edit_fields_list = fields[1:]
154         if table_name == "Payslips":
155             edit_fields_list = [x for x in edit_fields_list if x != "TotalOwed"]
156
157         entry_vars = {}
158         for i, col_name in enumerate(edit_fields_list, start=1):
159             Label(edit_win, text=col_name, bg="#87CEEB").pack(pady=5)
160             old_val = str(old_vals[i])
161
162             if table_name == "VehicleStock" and col_name == "VehicleCondition":
163                 var = StringVar(value=old_val)
164                 OptionMenu(edit_win, var, "Pristine", "Slightly Used", "Heavily Used").pac
165             elif table_name == "Customer" and col_name == "PreferredContactMethod":
166                 var = StringVar(value=old_val)
167                 OptionMenu(edit_win, var, "Email", "Phone", "SMS").pack(pady=5)
168             else:
169                 var = StringVar(value=old_val)
170                 Entry(edit_win, textvariable=var).pack(pady=5)
171
172             entry_vars[col_name] = var
173
174         Button(edit_win, text="Save Changes", command=save_edits,
175                bg="#4682B4", fg="white").pack(pady=20)
176
```



Pressing this button will start the edit_record() process.



For a record to be edited, a record must be clicked. To let the user know which record is selected, it will then be highlighted in blue as seen.



If the user attempts to edit a record without first selecting a record, this error message will pop up.



Similar to adding a record, a new window will pop up. However, this time the form is filled in allowing the user to make quick adjustments such as fixing a typo.

Delete a record

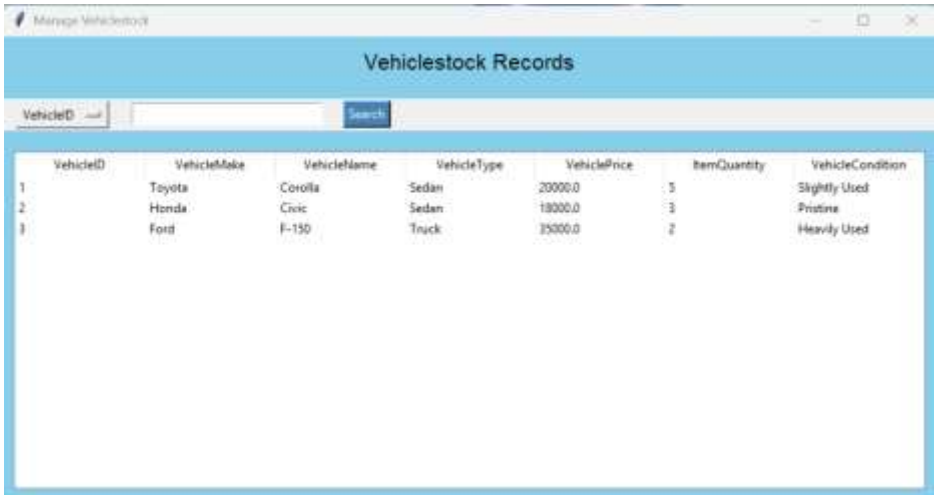
```
177     def delete_record():
178         sel = table_tree.selection()
179         if not sel:
180             messagebox.showerror("No Selection", "Select a record to delete.")
181             return
182         valz = table_tree.item(sel)["values"]
183         pk2 = valz[0]
184         ask = messagebox.askyesno("Confirm Delete", f"Delete record ID={pk2}?")
185         if ask:
186             conn = sqlite3.connect('mulligan_motors.db')
187             cursor = conn.cursor()
188             cursor.execute(f"DELETE FROM {table_name} WHERE {fields[0]} = ?", (pk2,))
189             conn.commit()
190             conn.close()
```



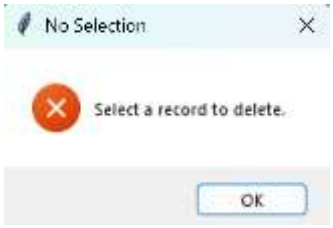
Pressing this delete button will begin the delete_record() process



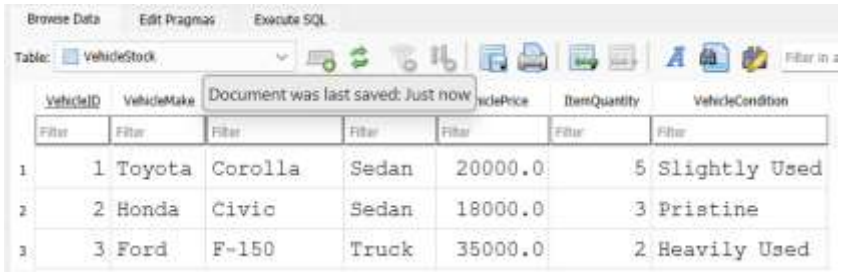
If a record is selected, a message box will open confirming the ID of the record to be deleted.



We can see that the targeted record has been successfully removed form the system.



If the user forgets to select a record to delete by clicking on it, this message box will appear instead.



The delete function is proven to have worked as seen within the database file itself. The targeted record is gone.

Automatic calculation of 'total owed'



```
if table_name == "Payslips":
    st_id = record_data["StaffID"]
    hrs2 = float(record_data["HoursWorked"])
    rate2 = float(record_data["HourlyRate"])
    tot2 = hrs2 * rate2
    conn = sqlite3.connect('mulligan_motors.db')
    cursor = conn.cursor()
    cursor.execute(update_query, (st_id, hrs2, rate2, tot2, pk))
    conn.commit()
    conn.close()
```

In the 'payslips' form, the user can input a staff members hourly rate and the hours they have worked. Calculating the 'total owed' manually field was cumbersome, unprofessional and prone to human error. Resultingly, I decided to calculate it automatically in the program.



The user isn't given the option to fill out the 'total owed' field.

When the record is saved, it will be automatically calculated ('HoursWorked' * 'HourlyRate') and stored.



PayslipID	StaffID	HoursWorked	HourlyRate	TotalOwed
1	1	40.0	20.0	800.0
2	2	35.0	15.0	525.0

As we can see in the treeview of the program, the total owed field has been calculated in successfully.

Drop down boxes



```
    entry_vars = {}
    for i, col_name in enumerate(edit_fields_list, start=1):
        Label(edit_win, text=col_name, bg="#87CEEB").pack(pady=5)
        old_val = str(old_vals[i])

        if table_name == "VehicleStock" and col_name == "VehicleCondition":
            var = StringVar(value=old_val)
            OptionMenu(edit_win, var, "Pristine", "Slightly Used", "Heavily Used").pack(pady=5)
        elif table_name == "Customer" and col_name == "PreferredContactMethod":
            var = StringVar(value=old_val)
            OptionMenu(edit_win, var, "Email", "Phone", "SMS").pack(pady=5)
        else:
            var = StringVar(value=old_val)
            Entry(edit_win, textvariable=var).pack(pady=5)

        entry_vars[col_name] = var
```

To limit human error, I decided to include drop down boxes in my solution to Mulligan Motors in fields with few and expected options.



For example, in the attribute, 'PreferredContactMethod' the user is forced to select from 3 preset values in a list as seen to the left.

Tree view

```
215     # Table tree
216     table_tree = ttk.Treeview(table_window, columns=fields, show="headings", height=15)
217     for col in fields:
218         table_tree.heading(col, text=col)
219         table_tree.column(col, width=120)
220     table_tree.pack(fill="x", padx=10, pady=10)
```

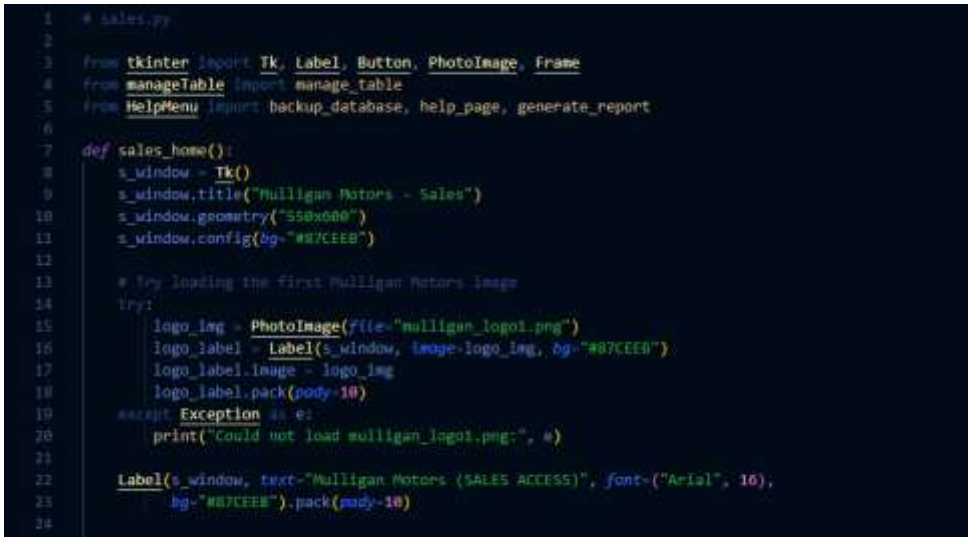


The code for the treeview function, allowing the user to view records, is seen above.

Sales.py

The sales file manages the framework for a user with ‘sales access’ which is meant to simulate the needs of a low-level employee in Muligan Motors.

Heading + Logo



```
# sales.py

from tkinter import Tk, Label, Button, PhotoImage, Frame
from manageTable import manage_table
from HelpMenu import backup_database, help_page, generate_report

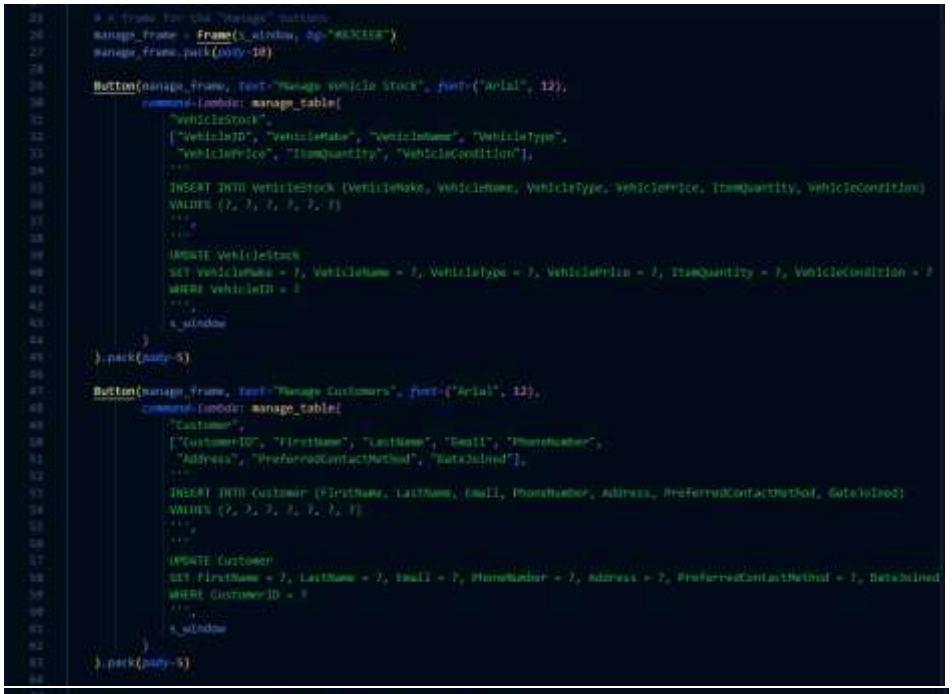
def sales_home():
    s_window = Tk()
    s_window.title("Mulligan Motors - Sales")
    s_window.geometry("550x600")
    s_window.config(bg="#87CEEB")

    # Try loading the first Mulligan Motors image
    try:
        logo_img = PhotoImage(file="mulligan_logo1.png")
        logo_label = Label(s_window, image=logo_img, bg="#87CEEB")
        logo_label.image = logo_img
        logo_label.pack(pady=10)
    except Exception as e:
        print("Could not load mulligan_logo1.png:", e)

    Label(s_window, text="Mulligan Motors (SALES ACCESS)", font=("Arial", 16),
          bg="#87CEEB").pack(pady=10)

```

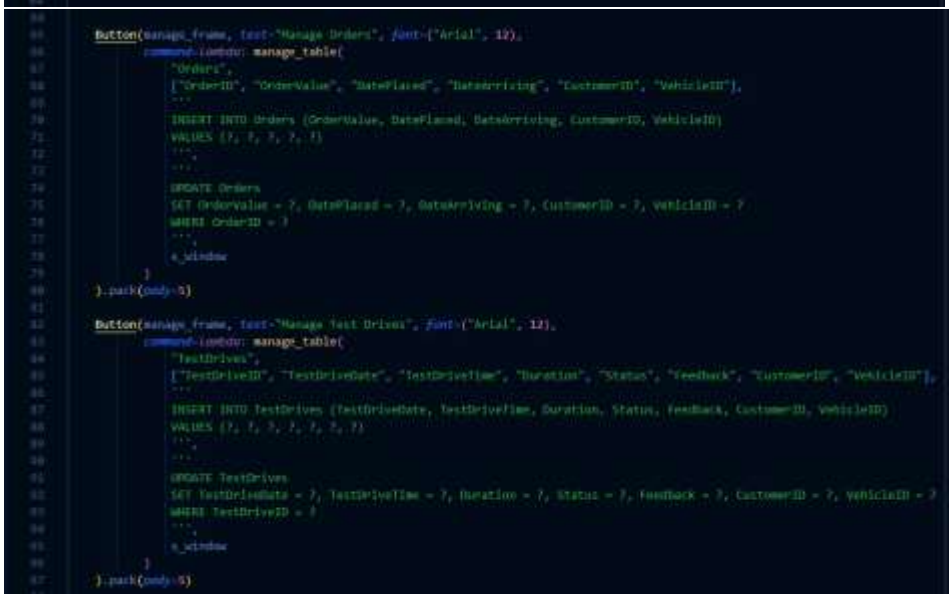
Manage table buttons



```
# A frame for the "Manage" buttons
manage_frame = Frame(s_window, bg="#E5C1CD")
manage_frame.pack(pady=10)

Button(manage_frame, text="Manage Vehicle Stock", font=("Arial", 12),
       command=lambda: manage_table(
           "VehicleStock",
           ["VehicleID", "VehicleMake", "VehicleName", "VehicleType",
            "VehiclePrice", "ItemQuantity", "VehicleCondition"],
           """
           INSERT INTO VehicleStock (VehicleMake, VehicleName, VehicleType, VehiclePrice, ItemQuantity, VehicleCondition)
           VALUES (?, ?, ?, ?, ?, ?)
           """,
           """
           UPDATE VehicleStock
           SET VehicleMake = ?, VehicleName = ?, VehicleType = ?, VehiclePrice = ?, ItemQuantity = ?, VehicleCondition = ?
           WHERE VehicleID = ?
           """,
           s_window
       )
).pack(pady=5)

Button(manage_frame, text="Manage Customers", font=("Arial", 12),
       command=lambda: manage_table(
           "Customer",
           ["CustomerID", "FirstName", "LastName", "Email", "PhoneNumber",
            "Address", "PreferredContactMethod", "DateJoined"],
           """
           INSERT INTO Customer (FirstName, LastName, Email, PhoneNumber, Address, PreferredContactMethod, DateJoined)
           VALUES (?, ?, ?, ?, ?, ?, ?)
           """,
           """
           UPDATE Customer
           SET FirstName = ?, LastName = ?, Email = ?, PhoneNumber = ?, Address = ?, PreferredContactMethod = ?, DateJoined
           WHERE CustomerID = ?
           """,
           s_window
       )
).pack(pady=5)
```



```
Button(manage_frame, text="Manage Orders", font=("Arial", 12),
       command=lambda: manage_table(
           "Orders",
           ["OrderID", "OrderValue", "DatePlaced", "DateArriving", "CustomerID", "VehicleID"],
           """
           INSERT INTO Orders (OrderValue, DatePlaced, DateArriving, CustomerID, VehicleID)
           VALUES (?, ?, ?, ?, ?)
           """,
           """
           UPDATE Orders
           SET OrderValue = ?, DatePlaced = ?, DateArriving = ?, CustomerID = ?, VehicleID = ?
           WHERE OrderID = ?
           """,
           s_window
       )
).pack(pady=5)

Button(manage_frame, text="Manage Test Drives", font=("Arial", 12),
       command=lambda: manage_table(
           "TestDrives",
           ["TestDriveID", "TestDriveDate", "TestDriveTime", "Duration", "Status", "FeedBack", "CustomerID", "VehicleID"],
           """
           INSERT INTO TestDrives (TestDriveDate, TestDriveTime, Duration, Status, FeedBack, CustomerID, VehicleID)
           VALUES (?, ?, ?, ?, ?, ?, ?)
           """,
           """
           UPDATE TestDrives
           SET TestDriveDate = ?, TestDriveTime = ?, Duration = ?, Status = ?, FeedBack = ?, CustomerID = ?, VehicleID = ?
           WHERE TestDriveID = ?
           """,
           s_window
       )
).pack(pady=5)
```

Utility Buttons frame

```
# A frame for utility buttons (Generate Report, Backup, Help)
utility_frame = Frame(s_window, bg="#87CEEB")
utility_frame.pack(pady=10)

Button(utility_frame, text="Generate Report", font=("Arial", 12),
       command=lambda: generate_report(s_window),
       bg="#2ecc71", fg="white").pack(pady=5)

Button(utility_frame, text="Backup Database", font=("Arial", 12),
       command=backup_database, bg="#468284", fg="white").pack(pady=5)

Button(utility_frame, text="Help", font=("Arial", 12),
       command=help_page, bg="#468284", fg="white").pack(pady=5)

# Finally, Log Off at the bottom
Button(s_window, text="Log Off", font=("Arial", 12),
       command=s_window.destroy, bg="#f39c12", fg="white").pack(side="bottom", pady=20)

s_window.mainloop()
```



This is the homescreen for those with ‘sales access’. Functionally, the system is the exact same. However, the access to tables is much smaller, e.g no ‘payslips’ table to prevent unauthorised access to important company data.

6.3 Comparison to success criteria

1. ...taken full account of the revised detailed design and refined the prototype in light of feedback and changed designs improvements into the system from feedback.

From looking at my code, it is evident that I have fulfilled all the criteria outlined in my investigation and design documents. I have also incorporated several improvements based on feedback received during prototyping, such as drop down boxes, refining the system to better align with user needs and revised designs.

2. .... produced a functioning solution to a highly demanding problem that meets most of the objectives for the solution to the problem. Better solutions will meet almost all of the objectives for the proposed solution.

I believe that my code provides a practical and functional program tailor built to Mulligan Motor's needs. My code meets most of the objectives outlined in the investigation and discussion documents, ensuring that the system operates as intended and fulfils its purpose effectively.

3. ...used and fully exploited the programming facilities of the language.

In developing this solution, I made full use of Python's programming facilities to create a functional and maintainable application. I chose to use Python's standard libraries, such as tkinter for the GUI and sqlite3 for database management, because they are well-suited for this type of application and allowed me to implement features efficiently. While I didn't heavily use classes or lambda functions, I prioritized clarity and functionality, ensuring the code is straightforward and easy to maintain. My design choices reflect a practical understanding of Python's strengths and the requirements of the project.

4. ...demonstrated a sound understanding of the appropriate tools and techniques available to them.

In my solution, I demonstrated a sound understanding of the appropriate tools and techniques by leveraging Python's capabilities effectively. I used tkinter to create an intuitive and responsive graphical user interface, ensuring a seamless user experience. For database management, I utilized sqlite3, which allowed me to design a relational database with proper normalization and foreign key constraints, ensuring data integrity and efficiency. Additionally, I employed Python's exception handling to manage errors gracefully, particularly during database operations and file I/O. By combining these tools and techniques, I ensured the application is robust, user-friendly, and well-structured, reflecting a strong grasp of the tools available in Python.

5. ...created a well-structured data model that aids efficient data handling. Better candidates will present a data model normalised to third normal form.

In designing the data model, I ensured it is well-structured and normalized to third normal form (3NF) to promote efficient data handling. Each table, such as VehicleStock, Customer, and Orders, is designed with a single responsibility, eliminating redundancy. For example:

- ✓ Foreign keys (e.g., CustomerID in Orders, StaffID in Access) enforce relationships between tables while avoiding data duplication.
- ✓ Atomic fields (e.g., splitting customer names into FirstName and LastName) adhere to 1NF.
- ✓ No transitive dependencies exist in tables like Payslips, where TotalOwed is derived directly from HoursWorked and HourlyRate without relying on non-key attributes.

By using PRAGMA foreign_keys = ON and cascading delete/set null rules, I maintained referential integrity. This design ensures efficient queries, minimizes redundancy, and keeps the data model scalable.

6. ...produced a solution that is well-structured and modular in nature. The solution makes good use of local variables and minimises the use of global variables

I structured the solution modularly by splitting it into multiple files (e.g., one for each of the databases core function), all linked through the main menu. I strictly avoided global variables, using local variables and function parameters to keep data isolated within their relevant scopes. Functions like manage_table handle specific operations independently, ensuring no shared state between modules. This design promotes reusability, simplifies debugging, and maintains a clean separation of concerns, making the codebase organized and easy to scale.

7. ...written code that is fully self-documenting and well-structured with annotation that will allow a competent third party to maintain the solution in future

I wrote the code to be self-documenting by using clear, descriptive names for functions, variables, and tables (e.g., validate_input, VehicleStock, PreferredContactMethod), ensuring that the logic is intuitive to follow. I added targeted annotations for complex sections, such as input validation rules in validate_input and database schema explanations in initialize_database. While I prioritized minimalism in comments, the modular structure, separation of concerns (e.g., splitting GUI and database logic), and consistent naming conventions make the codebase easy to navigate. A third party could maintain this solution confidently, as each function's purpose and data flow are evident from its design and context.

8. ...made use of complex user-defined routines. Better candidates will have made effective use of recursive algorithms.

I implemented complex user-defined routines, such as validate_input and manage_table, to handle detailed validation and dynamic database interactions. Recursion wasn't necessary because the tasks, like data validation and CRUD operations, were better suited

to iterative and modular approaches, which kept the code straightforward and efficient. The focus on reusable, well-structured functions ensured the solution remained robust and maintainable.

9. ...produced evidence of the effective use of validation for all key components. Better candidates will have created efficient routines for exception handling

In my solution, I implemented robust validation for all key components using the `validate_input` function, which enforces strict checks like required fields, numeric types (e.g., `VehiclePrice`), email formats, and foreign key integrity. For exception handling, I used `try-except` blocks in critical areas like database operations (`backup_database`, `generate_report`) to catch and gracefully handle errors, displaying user-friendly messages via `messagebox`. While recursion wasn't needed, this structured validation and error handling ensure data reliability and prevent crashes, even during edge-case scenarios.

10. ...fully documented the variables and actual data structures used to create the solution to the chosen problem

The data structure designs used in this system are documented in the design document. Below is a table listing all the variables used throughout the system. I believe this is evidence of the thorough documentation of the solution.

6.4 Variable list

1) DBSetup.py

Variable Name	Description	Data Type	Example Data
conn	Database connection object for mulligan_motors.db.	Database connection (sqlite3)	sqlite3.connect("mulligan_motors.db")
cursor	Cursor that executes SQL statements on the conn.	Database cursor	conn.cursor()
(local function)	The rest are local references inside initialize_database (e.g., table creation queries).	–	–

Notes:

- initialize_database() doesn't store persistent variables beyond conn and cursor; everything else is inline within queries.

2) InputValidation.py

Variable Name	Description	Data Type	Example Data
table_name	Name of the DB table being validated (e.g. "Customer", "Orders").	String	"VehicleStock"
data_dict	Dictionary containing user inputs for each field. We reference keys like "VehicleMake", "FirstName", etc.	Dictionary	{"FirstName": "Alice", "PhoneNumber": "1234"}

Variable Name	Description	Data Type	Example Data
"VehicleMake", "FirstName", etc.	**Dictionary keys** used to check presence or numeric/email validation.	(keys)	"VehicleMake": "Toyota"
is_float	Helper function to see if a string can convert to float.	Function	–
is_int	Helper function to see if a string can convert to int.	Function	–
email_pattern	Regex pattern for validating email addresses.	String (regex)	"^[\\w\\.-]+@[\\w\\.-]+\\.\\w+$"

Notes:

- validate_input() returns (True, "") or (False, "Error message").
- The dictionary **keys** are put in **quotes** if you need to list them in a table (e.g., "PhoneNumber").

3) HelpMenu.py

Variable Name	Description	Data Type	Example Data
help_window	Toplevel window for displaying help info.	Tkinter Toplevel	–
backup_database()	Function that prompts user for a file path and copies the local DB file.	Function	–

Variable Name	Description	Data Type	Example Data
generate_report()	Function that lets the user pick a table and save rows to .txt.	Function	_
"VehicleStock", "Customer", etc.	Table **names** in the report dictionary or drop-down.	(keys / strings)	"Orders", "Staff", etc.
file_path	Path chosen by user in asksaveasfilename for backup or report.	String (file path)	"C:/Backups/mulligan_backup.db"
rows	DB query results used to generate a .txt report.	List of tuples	[(1, 'Toyota', ...), (2, 'Honda', ...), ...]

Notes:

- help_page() uses local labels and text to display instructions.
- The dictionary entries for **report** are primarily the table names in the drop-down ("VehicleStock", "Staff", etc.).

4) manageTable.py

Variable Name	Description	Data Type	Example Data
table_name	Name of the table being managed, e.g. "Customer".	String	"Orders"

Variable Name	Description	Data Type	Example Data
fields	List of column names, e.g. ["CustomerID", "FirstName", "LastName", ...].	List of strings	["VehicleID","VehicleMake","VehicleName","VehicleType"]
insert_query	SQL INSERT statement for this table's new records.	String (SQL)	"INSERT INTO Customer (FirstName, LastName, ...) VALUES (...)"
update_query	SQL UPDATE statement for editing.	String (SQL)	"UPDATE Orders SET OrderValue=?, DatePlaced=? WHERE OrderID=?"
parent_window	Reference to the manager/sales window that spawned it.	Tkinter window	m_window or s_window
table_window	A new Toplevel showing the table records (search,	Tkinter Toplevel	–

Variable Name	Description	Data Type	Example Data
	add, edit, delete).		
dropdown_var	StringVar holding which column to search by.	StringVar (Tkinter)	"FirstName", "VehicleName"
search_entry	Entry where the user types the search term.	Tkinter Entry	–
table_tree	Treeview used to display the DB rows.	Tkinter Treeview	–
"record_data"	Dictionary of field -> user input when adding/editing a record.	Dictionary	{"FirstName": "Alice","PhoneNumber":"1234","Email":"a@b.com"}
"VehicleMake", "PhoneNumber" etc.	Dictionary keys inside "record_data".	(keys / strings)	"VehicleMake": "Toyota"

Notes:

- The user might see references like record_data[ff] or entry_vars[ff].
- This function also calls validate_input(table_name, record_data).

5) manager.py

Variable Name	Description	Data Type	Example Data
m_window	Main Tk() window for the Manager (admin) interface.	Tk window	–
logo2_img	PhotoImage loaded from "mulligan_logo2.png".	Tkinter PhotoImage	–
manage_frame	An optional Frame grouping “Manage” buttons.	Tkinter Frame	–
(Buttons)	Each Button calls manage_table(...) with relevant queries/fields for that table (Stock, Staff, etc).	Tkinter Button	–
(Utility Buttons)	“Generate Report,” “Backup Database,” “Help.”	Tkinter Button	–
(Log Off Button)	Button that closes the manager window via m_window.destroy().	Tkinter Button	–

Notes:

- None of these are formal dictionaries, but manager_home uses function calls to open CRUD windows for each table.

6) sales.py

Variable Name	Description	Data Type	Example Data
s_window	Main Tk() window for the Sales interface.	Tk window	–
logo_img	PhotoImage loaded from "mulligan_logo1.png".	Tkinter PhotoImage	–
(Buttons)	Each calls manage_table(...) for limited tables (VehicleStock, Customers...).	Tkinter Button	–

Variable Name	Description	Data Type	Example Data
(Utility Buttons)	“Generate Report,” “Backup Database,” “Help.”	Tkinter Button	–
(Log Off Button)	Button that closes s_window (s_window.destroy()).	Tkinter Button	–

Notes:

- Like manager.py, no major dictionaries are stored, just references to PhotoImage and Button objects.

7) LoginPage.py

Variable Name	Description	Data Type	Example Data
login_window	The main Tk() window for the login interface.	Tk window	–
logo1_img	PhotoImage loaded from "mulligan_logo1.png".	Tkinter PhotoImage	–
username_var	StringVar for the typed-in username.	Tkinter StringVar	"AliceUser"
password_var	StringVar for the typed-in password.	Tkinter StringVar	"mypassword"
"StaffID", "Username", "Password", "AccessLevel"	**Dictionary keys** in create_account_window for a new record in “Access”.	(keys / strings)	"Username": "NewUser", "Password": "abc123"
create_win	Toplevel used by create_account_window to gather new account info.	Tkinter Toplevel	–

Variable Name	Description	Data Type	Example Data
(Local function) validate_login()	Checks DB Access table to see if (username, password) is valid.	Function	–

Notes:

- create_account_window() uses a dictionary approach for checking uniqueness but is mostly local variables and DB calls.
- The actual dictionary insert is done with cursor.execute(...), passing separate parameters.

8) main.py

Variable Name	Description	Data Type	Example Data
initialize_database	Imported function that sets up all the DB tables.	Function	–
login_screen	Imported function from LoginPage.py that starts the UI.	Function	–

Notes:

- This file only calls initialize_database() then login_screen().
- No significant variables or dictionaries remain in scope.